JobSeekers Microservices - Docker & Kubernetes Deployment Guide

Table of Contents

1. Project Overview
2. Docker Setup
3. Kubernetes Basics
4. Minikube Deployment
5. Cloud Deployment Options
6. Troubleshooting

Project Overview

Project: JobSeekers Platform (Microservices Architecture)

Microservices (6):

- User-Service (Port 9020)
- Posts-Service (Port 9010)
- Notification-Service (Port 9040)
- Connections-Service (Port 9030)
- Discovery-Server/Eureka (Port 8761)
- API-Gateway (Port 8080)

Databases:

- PostgreSQL (notification-db, posts-db, user-db)

- Neo4j (connections-db)
- Kafka (Message Broker)

Technology Stack:

- Spring Boot (Java)
- Docker (Containerization)
- Kubernetes (Orchestration)
- Maven (Build Tool)

Docker Setup

Building Docker Images

Using Maven with Jib Plugin:

```bash
./mvnw clean package jib:build -DskipTests
```

This command:

1. Compiles Java code
2. Builds JAR file
3. Creates Docker image
4. Pushes to Docker Hub

For each microservice:

```bash
cd user-service
./mvnw clean package jib:build -DskipTests

cd ../post-service
./mvnw clean package jib:build -DskipTests

cd ../notification-service
./mvnw clean package jib:build -DskipTests

cd ../connections-service
./mvnw clean package jib:build -DskipTests

cd ../discovery-server
./mvnw clean package jib:build -DskipTests

cd ../api-gateway
./mvnw clean package jib:build -DskipTests
```

Docker Compose (Local Development)

```bash
docker-compose -f docker-compose.yml up -d
```

Common Docker Commands

```bash
```

```
# View running containers
docker ps

# View all containers
docker ps -a

# View images
docker images

# Stop all containers
docker stop $(docker ps -a -q)

# Remove all containers
docker rm $(docker ps -a -q)

# Remove all images
docker image prune -a -f

# View container logs
docker logs <container-name>

# Login to Docker Hub
docker login
```

Kubernetes Basics

What is Kubernetes (K8s)?

- Container orchestration platform
- Manages containerized applications

- Auto-scaling and load balancing
- 24/7 availability

Kubernetes Resources

Pod: Smallest unit (one or more containers)

Deployment: Manages multiple pod replicas

StatefulSet: For databases (maintains state)

Service: Exposes pods to network

Ingress: Routes HTTP traffic

kubectl Commands

```bash
```

```
# Get all resources
kubectl get pods
kubectl get svc
kubectl get deployments
kubectl get statefulsets

# Detailed view
kubectl describe pod <pod-name>

# View logs
kubectl logs <pod-name>
kubectl logs -f <pod-name>  # Follow logs

# Apply YAML file
kubectl apply -f <file.yml>

# Delete resources
kubectl delete pod <pod-name>
kubectl delete deployment <deployment-name>
kubectl delete all --all  # Delete everything

# Port forwarding
kubectl port-forward svc/service-name 8080:8080
```

Minikube Deployment

Prerequisites

- Docker Desktop installed
- Docker memory: 6GB+ allocated

- Minikube CLI installed

Starting Minikube

```bash
# Start with memory allocation
minikube start --memory=6144 --cpus=4

# Verify status
minikube status

# Get cluster info
kubectl cluster-info
```

Deployment Order (CRITICAL!)

Order matters because of dependencies:

```
1. Databases (no dependencies)
   - notification-db
   - posts-db
   - user-db
   - connections-db

2. Kafka (infrastructure)
   - kafka

3. Kafka-UI (depends on Kafka)
   - kafka-ui

4. Discovery-Server (Eureka)
   - discovery-server

5. Microservices (depend on databases + kafka)
   - user-service
   - post-service
   - notification-service
   - connections-service

6. API Gateway
   - api-gateway
```

Deployment Script

```bash
```

```
# Navigate to k8s folder
cd k8s

# Deploy databases
kubectl apply -f notification-db.yml
kubectl apply -f posts-db.yml
kubectl apply -f user-db.yml
kubectl apply -f connections-db.yml
sleep 15

# Deploy Kafka
kubectl apply -f kafka.yml
sleep 15

# Deploy Kafka-UI (optional)
kubectl apply -f kafkaui.yml
sleep 10

# Deploy Discovery Server
kubectl apply -f discovery-server.yml
sleep 10

# Deploy Microservices
kubectl apply -f user-service.yml
kubectl apply -f post-service.yml
kubectl apply -f notification-service.yml
kubectl apply -f connections-service.yml
sleep 10

# Deploy API Gateway
kubectl apply -f api-gateway.yml
```

Accessing Services

```bash
# Using minikube service command
minikube service api-gateway
minikube service discovery-server
minikube service kafka

# Get Minikube IP
minikube ip

# Manual access
http://<minikube-ip>:30XXX
```

Monitoring Deployment

```bash
# Watch pods in real-time
kubectl get pods -w

# Check all pods status
kubectl get pods

# Check services
kubectl get svc

# View pod logs
kubectl logs <pod-name>

# Get detailed pod info
kubectl describe pod <pod-name>
```

YAML Configuration Guide

Database StatefulSet (PostgreSQL)

```yaml
```

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: notification-db
spec:
  serviceName: notification-db
  replicas: 1
  selector:
    matchLabels:
      app: notification-db
  template:
    metadata:
      labels:
        app: notification-db
    spec:
      containers:
      - name: notification-db
        image: postgres:16
        env:
          - name: POSTGRES_USER
            value: postgres
          - name: POSTGRES_PASSWORD
            value: "456123456"
          - name: POSTGRES_DB
            value: notificationDB
          - name: PGDATA
            value: /var/lib/postgresql/data/pgdata
        ports:
        - containerPort: 5432
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
```

```yaml
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: "standard"
      resources:
        requests:
          storage: 1Gi

---
apiVersion: v1
kind: Service
metadata:
  name: notification-db
spec:
  clusterIP: None
  selector:
    app: notification-db
  ports:
  - port: 5432
    targetPort: 5432
```

Microservice Deployment

```yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
      - name: user-service
        image: yasiffkhan/jobseekers:user-service
        resources:
          limits:
            memory: "400Mi"
            cpu: "200m"
          requests:
            memory: "200Mi"
            cpu: "100m"
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: k8s
          - name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE
            value: http://discovery-server:8761/eureka
          - name: SPRING_DATASOURCE_URL
            value: jdbc:postgresql://user-db:5432/usersDB
          - name: SPRING_DATASOURCE_USERNAME
            value: postgres
```

```yaml
          - name: SPRING_DATASOURCE_PASSWORD
            value: "456123456"
          - name: KAFKA_BOOTSTRAP_SERVERS
            value: kafka:9092
        ports:
        - containerPort: 9020
        livenessProbe:
          httpGet:
            path: /actuator/health
            port: 9020
          initialDelaySeconds: 60
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health
            port: 9020
          initialDelaySeconds: 30
          periodSeconds: 10

---
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  type: ClusterIP
  selector:
    app: user-service
  ports:
  - port: 9020
    targetPort: 9020
```

Cloud Deployment Options

Option 1: Oracle OKE (FREE) ✅ RECOMMENDED

Pros:

- Completely free tier
- No card charges if within limits
- 1 Kubernetes cluster + resources
- Always-free (no expiration)

Setup:

```bash
# Create free account
# https://www.oracle.com/cloud/free

# Install Oracle CLI
choco install oci-cli  # Windows

# Configure
oci setup config

# Create cluster
oci ce cluster create --name jobseekers-cluster \
  --kubernetes-version v1.27 \
  --compartment-id <your-id>
```

Option 2: Google GKE

Pros:

- Professional Kubernetes service
- Good support

Cons:

- Requires credit card
- ₹500+/month cost

Option 3: AWS EKS

Pros:

- Industry standard
- Scalable

Cons:

- Requires credit card
- ₹500+/month cost

Option 4: Heroku (Non-Kubernetes) ✅ EASIEST

Pros:

- Completely free
- No card needed
- Easy deployment
- Perfect for portfolio

Setup:

```bash
heroku login
heroku create your-app-name
git push heroku main
```

Option 5: Railway.app

Pros:

- Free credits ($5/month)
- Easy deployment

Cons:

- Credits run out

Troubleshooting

Pod keeps restarting (CrashLoopBackOff)

```bash
```

```
# Check logs
kubectl logs <pod-name>

# Check pod details
kubectl describe pod <pod-name>

# Common causes:
# - Missing environment variables
# - Database connection failed
# - Port already in use
# - Insufficient memory
```

Can't connect to database

```bash
# Verify database is running
kubectl get pods | grep db

# Check database service name
kubectl get svc

# Test connection
kubectl port-forward svc/notification-db 5432:5432
```

Kafka keeps crashing

Cause: Insufficient memory

Solution:

```bash
```

```
minikube stop
minikube delete
minikube start --memory=8192 --cpus=4
```

Service not accessible

```bash
# Verify service exists
kubectl get svc

# Port forward for testing
kubectl port-forward svc/api-gateway 8080:8080

# Access at http://localhost:8080
```

Insufficient disk space

```bash
# Clean up Docker
docker image prune -a -f
docker container prune -f

# Stop Minikube
minikube stop
minikube delete
```

Common Issues & Solutions

Environment Variable Errors

WRONG:

```yaml
value: 456123456  # Treated as number
```

CORRECT:

```yaml
value: "456123456"  # String
```

Service Communication

Use service name (DNS):

```yaml
SPRING_DATASOURCE_URL: jdbc:postgresql://notification-db:5432/notificationDB
KAFKA_BOOTSTRAP_SERVERS: kafka:9092
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka
```

Storage Class Issues

Minikube uses "standard":

```yaml
storageClassName: "standard"  # NOT "standard-rwo" (GKE only)
```

Performance Tips

For Minikube

- Allocate 6GB+ RAM to Docker Desktop
- Allocate 4+ CPUs
- Use `--cpus=4 --memory=6144` flags
- Stop Minikube when not in use

For Production

- Use at least 2 replicas per service
- Set resource limits and requests
- Use health checks (liveness + readiness probes)
- Implement auto-scaling

Next Steps

1. **Local Development**
    - Use Docker Compose for fastest setup
    - Test locally before Kubernetes
2. **Learning Kubernetes**
    - Start with Minikube
    - Learn kubectl commands
    - Understand YAML syntax

3. **Production Deployment**
    - Choose cloud provider (Oracle OKE recommended)
    - Setup CI/CD pipeline
    - Monitor and scale

Quick Reference Commands

```bash
```

```
# Minikube
minikube start --memory=6144 --cpus=4
minikube stop
minikube delete
minikube ip
minikube service <service-name>

# kubectl
kubectl get pods
kubectl get svc
kubectl apply -f file.yml
kubectl delete pod <pod-name>
kubectl logs -f <pod-name>
kubectl describe pod <pod-name>
kubectl port-forward svc/<service> 8080:8080

# Docker
docker login
docker build -t image:tag .
docker push image:tag
docker ps
docker logs <container>
docker-compose up -d
docker-compose down

# Maven
./mvnw clean package -DskipTests
./mvnw clean package jib:build -DskipTests
```

Contact & Support

For issues or questions:

- Check kubectl describe output
- Review pod logs
- Verify YAML syntax
- Ensure all dependencies are running

Remember: Check pod dependencies before deploying!

Last Updated: February 2026 **Author:** Claude AI Assistant **Project:** JobSeekers Microservices Platform